



ChatBot
Aula 12

Prof. Me Daniel Vieira



Agenda

- 1 - Engenharia de prompt
- 2 - Chatbot com Gemini
- 3 - Exemplos
- 4 - Exercício

Engenharia de Prompt

Engenharia de prompt é a prática de estruturar instruções para que o modelo responda com mais qualidade, consistência e utilidade. Em vez de apenas “perguntar qualquer coisa”, você define papel, contexto, objetivo, regras e formato de resposta. Isso melhora previsibilidade e reduz respostas vagas. A documentação oficial do Gemini destaca justamente o uso de system instructions e structured outputs para orientar comportamento e formato

Engenharia de Prompt

1.2 Estrutura de um bom prompt

Uma estrutura simples que funciona bem em aula é:

Papel + Contexto + Tarefa + Restrições + Formato de saída

Exemplo ruim:

Fale sobre razão.

Engenharia de Prompt

Exemplo melhor:

Você é um atendente virtual de petshop.

Responda dúvidas de clientes sobre tipos de ração para cães e gatos.

Explique de forma simples, em português do Brasil.

Se a informação não estiver no catálogo, diga que não encontrou no sistema.

Responda em no máximo 5 linhas.

1.3 Técnicas de prompt para ensinar

Você pode mostrar estas técnicas com exemplos:

Prompt direto

Explique a diferença entre ração premium e super premium.

Engenharia de Prompt

Prompt com papel

Você é um veterinário explicando para um tutor iniciante.
Explique a diferença entre ração premium e super premium.

Engenharia de Prompt

Prompt com restrição

Explique a diferença entre ração premium e super premium em até 4 tópicos.

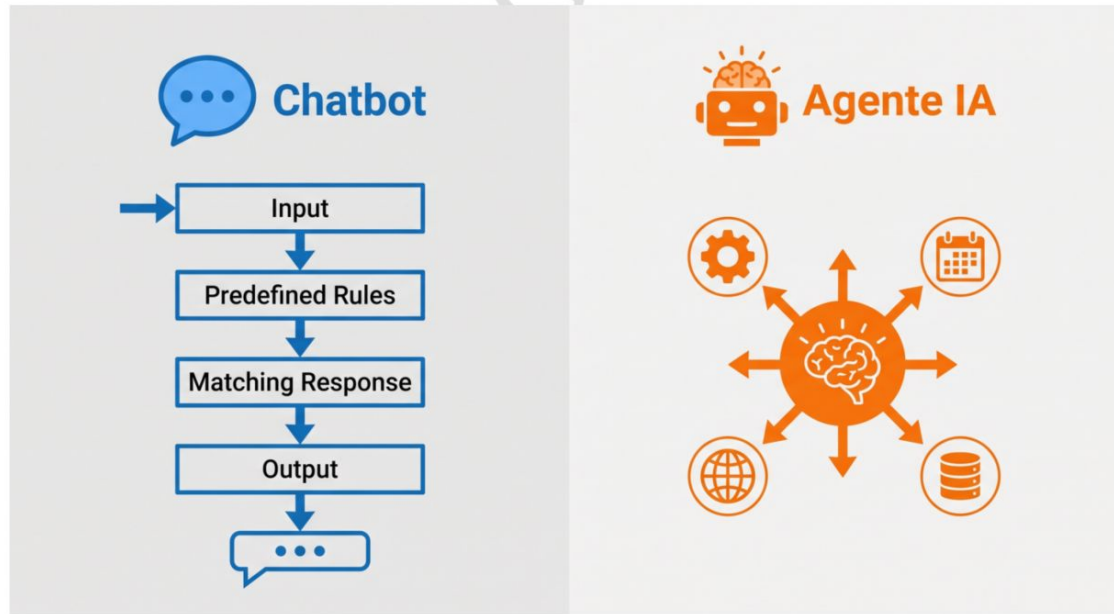
Engenharia de Prompt

Prompt com saída estruturada

Responda em JSON com os campos:
tipo, indicacao, beneficios

Saídas estruturadas com JSON Schema são oficialmente suportadas pelo Gemini e são ótimas para automação e integração com sistemas

Agente de IA



Agente de IA

Um chatbot comum responde perguntas.

Um agente, além de responder, usa regras, contexto e eventualmente ferramentas para tomar pequenas ações, consultar dados e devolver respostas mais úteis.

Agente de IA

Chatbots Tradicionais: Geralmente operam com base em regras e fluxos pré-definidos. São como um sistema de atendimento telefônico ("Digite 1 para financeiro, digite 2 para suporte..."). Eles são ótimos para responder a um conjunto limitado de perguntas previsíveis, mas falham quando o usuário desvia do script.

Agentes de IA: São um salto evolutivo. Um agente de IA, alimentado por um LLM como o Gemini, não segue apenas um script. Ele interpreta a intenção do usuário, cria um plano de ação para atingir um objetivo e pode até mesmo usar ferramentas externas (como acessar um banco de dados, consultar outra API ou executar um script) para completar sua tarefa.

Exemplos de prompt

Exemplo 1 — Prompt fraco

Fale sobre ração para cachorro.

Exemplo 2 — Prompt melhorado

Você é um atendente virtual de petshop.

Explique ao cliente quais são os tipos de ração para cachorro.

Use linguagem simples.

Organize em 3 tópicos:

1. filhote
2. adulto
3. idoso

Exemplos de prompt

Exemplo 3 — Prompt com regra de negócio

Você é um atendente virtual de petshop.

Responda apenas com base no catálogo enviado.

Nunca invente preço.

Se o produto não existir no catálogo, diga:

"Desculpe, não encontrei esse item no sistema."

Exemplos de prompt

Exemplo 4 — Prompt para agente

Você é um agente virtual de um petshop.

Sua função é ajudar clientes com dúvidas sobre ração, marcas, faixas de preço e indicação por porte do animal.

Use somente os dados do catálogo.

Se o cliente perguntar algo fora do catálogo, informe com educação que a loja precisa consultar um atendente humano.

Sempre responda em português do Brasil.

Projeto Agente de IA para PetShop

Um agente que:

recebe perguntas do cliente;

consulta um catálogo local de produtos;

envia contexto para o Gemini;

devolve respostas sobre ração e valores

Projeto Agente de IA para PetShop

Um agente que:

recebe perguntas do cliente;

consulta um catálogo local de produtos;

envia contexto para o Gemini;

devolve respostas sobre ração e valores

Código Agente

```
1 !pip install -U google-genai # instala biblioteca do google de ia generativa
```

```
1 import os # biblioteca para acessar variaveis de ambiente  
2 from google.colab import userdata # importa o userdata para carregar variaveis
```

```
1 os.environ["GEMINI_API_KEY"] = userdata.get("GEMINI_API_KEY")
```

Código Agente

```
# Importa a biblioteca os para acessar variáveis de ambiente
import os

# Importa json para transformar listas e dicionários em texto JSON
import json

# Importa o cliente da API Gemini
from google import genai

# Importa tipos de configuração da biblioteca Gemini
from google.genai import types
```

Código Agente

```
# Catálogo de produtos do petshop
catalogo_petshop = [
    {
        "nome": "Ração Golden Fórmula Cães Adultos 15kg",
        "categoria": "ração para cachorro",
        "porte": "adulto",
        "marca": "Golden",
        "preco": 149.90
    },
```

Código Agente

```
{  
  "nome": "Ração Premier Fórmula Cães Filhotes 10kg",  
  "categoria": "ração para cachorro",  
  "porte": "filhote",  
  "marca": "Premier",  
  "preco": 189.90  
},  
{  
  "nome": "Ração Whiskas Carne Gatos Adultos 10kg",  
  "categoria": "ração para gato",  
  "porte": "adulto",  
  "marca": "Whiskas",  
  "preco": 129.90  
},
```

Código Agente

```
{  
    "nome": "Ração Royal Canin Gatos Castrados 7.5kg",  
    "categoria": "ração para gato",  
    "porte": "adulto",  
    "marca": "Royal Canin",  
    "preco": 239.90  
}  
]
```

Código Agente

```
# Cria o cliente da API Gemini
client = genai.Client(api_key=os.getenv( "GEMINI_API_KEY" ))

def extrair_preco_maximo (pergunta):
    """
    Tenta encontrar um valor numérico na pergunta para usar como filtro de preço.
    Exemplo:
    'ração até 150 reais' -> 150.0
    """
    palavras = pergunta.replace( ",", "." ).split()

    for palavra in palavras:
        try:
            return float(palavra)
        except ValueError:
            continue
    return None
```

Código Agente

```
def buscar_produtos(pergunta):  
    """  
    Busca produtos no catálogo com regras mais inteligentes.  
    """  
    pergunta = pergunta.lower().strip()  
  
    # Se a pessoa pedir catálogo completo ou tipos de ração, retorna tudo  
    gatilhos_catalogo = [  
        "catalogo", "catálogo", "produtos", "mostrar tudo", "mostre tudo",  
        "todas as rações", "todas as racoes", "tipos de ração", "tipos de  
        racao",  
        "quais rações", "quais racoes", "quais produtos"  
    ]
```


Código Agente

```
if any(gatilho in pergunta for gatilho in gatilhos_catalogo):  
    return catalogo_petshop
```

```
resultados = []
```

Código Agente

```
# Filtros identificados na pergunta
quer_cachorro = any(p in pergunta for p in ["cachorro", "cão", "cao",
"cães", "caes"])
quer_gato = "gato" in pergunta or "gatos" in pergunta
quer_filhote = "filhote" in pergunta or "filhotes" in pergunta
quer_adulto = "adulto" in pergunta or "adultos" in pergunta

preco_maximo = None
if "até" in pergunta or "ate" in pergunta or "menos de" in pergunta:
    preco_maximo = extrair_preco_maximo(pergunta)
```

Código Agente

```
# Palavras comuns que não ajudam na busca
```

```
palavras_ignoradas = {  
    "de", "do", "da", "das", "dos", "para", "com", "sem", "e", "ou",  
    "a", "o", "as", "os", "um", "uma", "quero", "qual", "quais",  
    "tem", "temos", "você", "você", "ração", "rações", "races"  
}
```

```
palavras_busca = [p for p in pergunta.split() if p not in  
palavras_ignoradas]
```

Código Agente

```
for produto in catalogo_petshop:
    # Começa assumindo que o produto atende
    atende = True

    # Filtro por tipo de animal
    if quer_cachorro and "cachorro" not in produto["categoria"]:
        atende = False

    if quer_gato and "gato" not in produto["categoria"]:
        atende = False
```

Código Agente

```
# Filtro por porte/faixa
```

```
if quer_filhote and produto["porte"] != "filhote":  
    atende = False
```

```
if quer_adulto and produto["porte"] != "adulto":  
    atende = False
```

```
# Filtro por preço
```

```
if preco_maximo is not None and produto["preco"] > preco_maximo:  
    atende = False
```

Código Agente

```
# Busca por marca ou outras palavras relevantes
texto_produto = f"{produto['nome']} {produto['categoria']} {produto['porte']}
{produto['marca']}".lower()

# Se não houver filtros específicos, usa busca por palavras
if not (quer_cachorro or quer_gato or quer_filhote or quer_adulto or preco_maximo is
not None):
    if palavras_busca:
        if not any(palavra in texto_produto for palavra in palavras_busca):
            atende = False

    if atende:
        resultados.append(produto)

return resultados
```

Código Agente

```
def responder_cliente(pergunta):  
    """  
    Gera a resposta do agente com base nos produtos encontrados.  
    """  
    produtos_encontrados = buscar_produtos(pergunta)  
  
    contexto_catalogo = json.dumps(produtos_encontrados, ensure_ascii= False, indent=2)
```

Código Agente

```
prompt = f"""
```

```
Você é um agente virtual de um petshop.
```

```
Regras:
```

- Responda somente com base nos produtos encontrados no catálogo.
- Não invente produtos ou preços.
- Se não encontrar produtos, diga claramente que não encontrou no sistema.
- Se o cliente pedir o catálogo ou tipos de ração, apresente os produtos disponíveis.
- Responda em português do Brasil.
- Seja educado e objetivo.

```
Pergunta do cliente:
```

```
{pergunta}
```


Código Agente

Produtos encontrados:

```
{contexto_catalogo}
```

```
"""
```

```
response = client.models.generate_content(
```

```
    model="gemini-2.5-flash-lite",
```

```
    contents=prompt,
```

```
    config=types.GenerateContentConfig(
```

```
        system_instruction="Você é um atendente virtual especializado em produtos de
```

```
petshop."
```

```
    )
```

```
)
```

```
return response.text
```

Código Agente

```
# Loop principal
while True:
    pergunta = input("\nCliente: ")

    if pergunta.lower() in ["sair", "exit", "quit"]:
        print("Encerrando atendimento.")
        break

    resposta = responder_cliente(pergunta)
    print("\nAgente:", resposta)
```

Resposta Agente

Cliente: tipos de ração

celula de código

Agente: Olá! Temos os seguintes tipos de ração em nosso catálogo:

- * **Ração Golden Fórmula Cães Adultos 15kg** - Marca: Golden, Preço: R\$ 149,90
- * **Ração Premier Fórmula Cães Filhotes 10kg** - Marca: Premier, Preço: R\$ 189,90
- * **Ração Whiskas Carne Gatos Adultos 10kg** - Marca: Whiskas, Preço: R\$ 129,90
- * **Ração Royal Canin Gatos Castrados 7.5kg** - Marca: Royal Canin, Preço: R\$ 239,90

Posso ajudar com mais alguma informação sobre esses produtos?

Cliente: shampoo tem ?

Agente: No momento, não temos shampoos em nosso catálogo.

Cliente: ola

Agente: Olá! Seja bem-vindo ao nosso petshop virtual. Como posso ajudar você hoje?

Cliente: catalogo

Agente: Olá! Segue o nosso catálogo de produtos disponíveis:

Resposta Agente

Cliente: tipos de ração

celula de código

Agente: Olá! Temos os seguintes tipos de ração em nosso catálogo:

- * **Ração Golden Fórmula Cães Adultos 15kg** - Marca: Golden, Preço: R\$ 149,90
- * **Ração Premier Fórmula Cães Filhotes 10kg** - Marca: Premier, Preço: R\$ 189,90
- * **Ração Whiskas Carne Gatos Adultos 10kg** - Marca: Whiskas, Preço: R\$ 129,90
- * **Ração Royal Canin Gatos Castrados 7.5kg** - Marca: Royal Canin, Preço: R\$ 239,90

Posso ajudar com mais alguma informação sobre esses produtos?

Cliente: shampoo tem ?

Agente: No momento, não temos shampoos em nosso catálogo.

Cliente: ola

Agente: Olá! Seja bem-vindo ao nosso petshop virtual. Como posso ajudar você hoje?

Cliente: catalogo

Agente: Olá! Segue o nosso catálogo de produtos disponíveis:

Exercício

Crie um agente virtual para uma loja de autopeças utilizando a API do Google Gemini.

O agente deverá:

- responder perguntas sobre produtos da loja;
- informar preços apenas quando estiverem no catálogo;
- sugerir peças compatíveis com base no tipo de veículo, quando essa informação existir;
- nunca inventar valores ou disponibilidade;
- informar quando a dúvida precisar de atendimento humano.

Exercício

```
catalogo_autopeca = [
    {
        "nome": "Pastilha de Freio Bosch Fiat Uno",
        "categoria": "freio",
        "marca": "Bosch",
        "preco": 89.90,
        "veiculo": "Fiat Uno"
    },
    {
        "nome": "Filtro de Óleo Fram Gol 1.0",
        "categoria": "filtro de óleo",
        "marca": "Fram",
        "preco": 29.90,
        "veiculo": "VW Gol 1.0"
    },
    {
        "nome": "Bateria Moura 60Ah",
        "categoria": "bateria",
        "marca": "Moura",
        "preco": 399.90,
        "veiculo": "universal"
    }
]
```

Exercício

```
• },  
• {  
•   "nome": "Amortecedor Dianteiro Cofap Corsa",  
•   "categoria": "suspensão",  
•   "marca": "Cofap",  
•   "preco": 219.90,  
•   "veiculo": "Chevrolet Corsa"  
• }  
• ]
```

Exercício

Exemplos de perguntas para teste

- “Vocês têm bateria para carro?”
- “Quanto custa a pastilha de freio do Uno?”
- “Tem filtro para Gol 1.0?”
- “Qual amortecedor serve no Corsa?”
- “Quanto custa a correia dentada do Civic?”

Nesse último caso, o agente deve responder que não encontrou no sistema.

Obrigado!

Prof. Me Daniel Vieira

Email: danielvieira2006@gmail.com

Linkedin: Daniel Vieira

Instagram: Prof daniel.vieira95

